Pointillismus

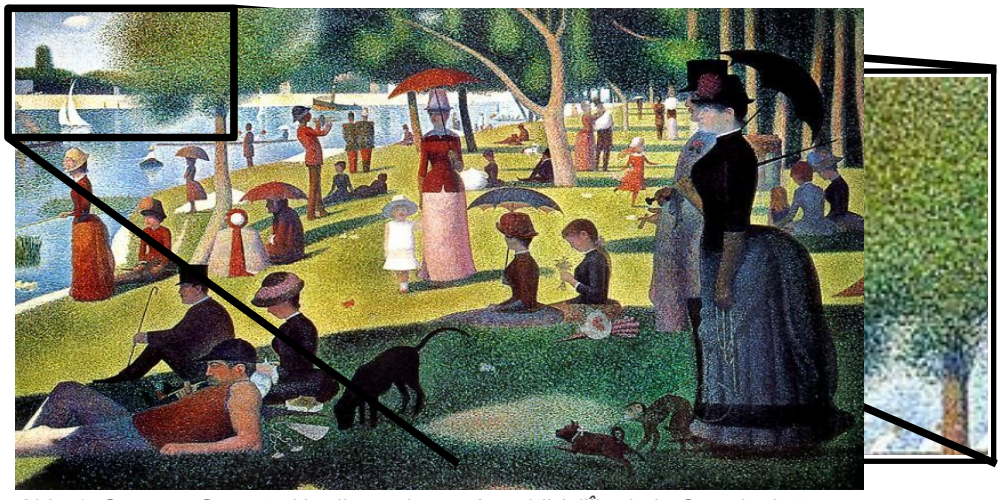
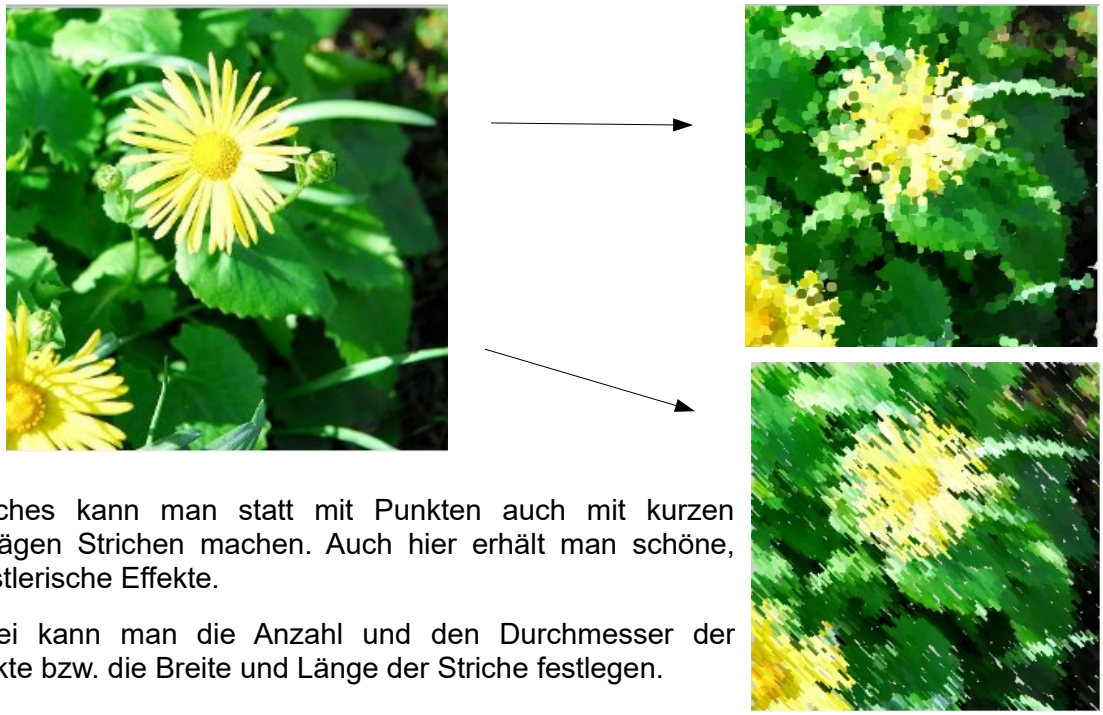


Abb. 1: Georges Seurat - Un dimanche après-midi à l'Île de la Grande Jatte Wikimedia Commons (Lizenz: Public Domain), URL: https://commons.wikimedia.org/wiki/File:Georges_Seurat_-_Un_dimanche_apr%C3%A8s-midi_%C3%A0_l%27%C3%8Ele_de_la_Grande_Jatte.jpg

Pointillismus bezeichnet eine Stilrichtung in der Malerei. Sie hatte in den Jahren zwischen *1889* und *1910* ihre Blütezeit. Der Pointillismus wird dem Post-Impressionismus zugeordnet.

Typisch für den Pointillismus ist der streng geometrisch durchkomponierte, oft ornamental wirkende Bildaufbau. Die Farben werden dabei in kleinen Punkten auf die Leinwand gesetzt. Dabei nutzten pointillistische Maler nur reine Farben. Farbmischungen entstehen erst auf dem Malgrund selbst.

Man kann einen ähnlichen Effekt erreichen, wenn man in einem Bild zufällige Pixel auswählt und dann ausgefüllte Kreise einer bestimmten Größe an diese Stelle zeichnet.



Gleiches kann man statt mit Punkten auch mit kurzen schrägen Strichen machen. Auch hier erhält man schöne, künstlerische Effekte.

Dabei kann man die Anzahl und den Durchmesser der Punkte bzw. die Breite und Länge der Striche festlegen.

Erweiterung: Stricheln / Tupfen



Zufallszahlen in Java

Um die Pixel zufällig auszuwählen, benötigt man Zufallszahlen. In Java gibt es eine Klasse Random, um Zufallszahlen zu erzeugen. Möchte man Sie benutzen, muss man sie importieren und dann ein Random-Objekt erzeugen. Bei diesem gibt es verschiedene Methoden, um reelle oder ganzzahlige Zufallszahlen zu erzeugen.

In Processing ist es etwas einfacher:

```
int x = int(random(originalbild.width));
```

erzeugt eine ganzzahlige Zufallszahl zwischen 0 und der Breite des Originalbildes.

Wiederholung: Zeichnen mit Processing

Die folgenden Zeichenmethoden kennst du schon aus Klasse 9:

```
stroke(250,0,0);       // Stiftfarbe rot
strokeWeight(3);       // Stiftbreite 3
line(10,10,30,10);     // Linie von (10,10) nach (30,10)
noStroke();            // kein Stift/Rand
fill(0,250,0);         // Füllfarbe grün
ellipse(20,20,10,10); // Kreisfläche an (20,20) mit d=10
```

Aufgaben

1. *Implementiere eine Methode*
   `PImage tupfen(PImage originalbild, int groesse, int anzahl)`
   Wähle eine zufällige x-Koordinate und eine zufällige y-Koordinate auf dem Bild aus. Setze die Füllfarbe auf die Farbe des Pixels im Originalbild und zeichne auf das Bild eine Kreisfläche (ohne Rand) an der ausgewählten Stelle mit dem Durchmesser `groesse`*. Wiederhole das in einer Schleife* `anzahl` *mal. Erzeuge danach ein neues Bild von der Größe des Originalbildes und übergebe jedem Pixel (wie gewohnt in einer verschachtelten Schleife) das Pixel der veränderten Oberfläche mit* `get(x,y)`*.*
   Teste deine Methode.
2. *Deklariere analog eine Methode* `stricheln` *mit den Parametern* `originalbild`*,* `dicke`*,* `laenge` *und* `anzahl`*.*
   Zeichne die gewünschte Anzahl von Strichen auf das neue Bild. Wähle auch hier die Positionen zufällig und benutze die Farbe des Originalbildes an dieser Stelle. Die Stiftbreite muss auf `dicke` *eingestellt werden.*
   Es sieht schöner aus, wenn die Striche schräg sind. Überlege dir, wie du den Endpunkt einer schrägen Strecke berechnen kannst, deren Länge über den Parameter verändert werden kann.

3. *Ergänze deine GUI so, dass man die nötigen Angaben für die beiden neuen Methoden über Textfelder eingeben und dann das geladene Bild tupfen oder stricheln lassen kann. (Info: Zahlen in Textfeldern gelten als String/Zeichenkette. Sie müssen umgewandelt werden. Z.B.* `int(tfStrichanzahl.getText())`*. Ist keine Zahl im Feld, erzeugt man einen Fehler.)*